

Proyecto: Permutation Flow Shop Scheduling Problem (PFSP)

Resolución mediante Algoritmos Evolutivos

1. Introducción y motivación

Imagina una fábrica textil donde cada pedido (trabajo) debe pasar, en el **mismo orden**, por una serie de estaciones: corte → costura → planchado → empaquetado. Cada estación es una **máquina**, y cada trabajo tarda un tiempo distinto en cada una de ellas.

La pregunta que resuelve el **Permutation Flow Shop Scheduling Problem (PFSP)** es:

¿En qué **orden** debemos secuenciar los trabajos para que la fábrica termine todos lo antes posible?

Este tipo de problema aparece en manufactura, líneas de ensamblaje, procesamiento de datos por lotes (pipelines), impresión, e incluso en programación de tareas en sistemas computacionales. Es uno de los problemas de *scheduling* más estudiados en optimización combinatoria, y su espacio de búsqueda crece como $n!$, lo que lo hace intratable por fuerza bruta para instancias medianas o grandes, y por eso es un candidato ideal para metaheurísticas.

2. Definición formal del problema

- Hay **n trabajos** (jobs): $J_1, J_2, \dots, J_n$
- Hay **m máquinas**: $M_1, M_2, \dots, M_m$
- **Todos los trabajos pasan por todas las máquinas en el mismo orden**: primero M_1 , luego M_2 , ..., hasta M_m . Esto es lo que distingue al *flow shop* de otros problemas de *scheduling* (como el *job shop*, donde cada trabajo puede tener su propio orden de máquinas — eso NO es lo que resolveremos aquí).
- Se conoce de antemano el **tiempo de procesamiento** $p_{i,j}$ = tiempo que tarda el trabajo i en la máquina j .
- **Restricción clave (lo que hace "Permutation" al problema)**: el orden en que los trabajos entran a la máquina 1 debe ser **el mismo orden** en que entran a todas las demás máquinas. Es decir, no se pueden "adelantar" ni "atrasar" trabajos entre estaciones — todos siguen la misma fila.

Objetivo

Encontrar la **permutación de los n trabajos** $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ que **minimiza el makespan** (C_{max}): el tiempo en que **termina el último trabajo en la última máquina**.

Nota de notación (para no confundir J con π): J_i es el trabajo número i como entidad fija — trae consigo sus propios tiempos de procesamiento $p_{i,j}$, que nunca cambian. π , en cambio, es el **orden** que estamos buscando: π_k significa "el trabajo que ocupa la posición k en la secuencia". Por ejemplo, si $\pi = (J_3, J_1, J_2)$, entonces $\pi_1 = J_3$ (en la posición 1 va el trabajo 3), $\pi_2 = J_1$, y $\pi_3 = J_2$. El cromosoma del AG (sección 4) es exactamente π escrito como un arreglo de índices de trabajos.

3. Ejemplo numérico paso a paso (para que quede 100% claro)

Supongamos una instancia pequeña con **3 trabajos y 2 máquinas**, con los siguientes tiempos de procesamiento:

Trabajo	Máquina 1 (M1)	Máquina 2 (M2)
J1	5	3
J2	2	6
J3	4	4

Probemos la secuencia $\pi = (J1, J2, J3)$.

Regla de cálculo: cada trabajo puede empezar en una máquina recién cuando (a) esa máquina quedó libre del trabajo anterior, Y (b) el propio trabajo ya terminó en la máquina previa.

$$C(i, j) = \max(C(i-1, j), C(i, j-1)) + p_{i,j}$$

Construimos la tabla de tiempos de finalización $C(i, j)$:

Máquina 1 ($M1$ ó $j = 1$): cada trabajo solo espera a que $M1$ quede libre del anterior.

$$C(J1, M1) = C(1, 1) = \max(C(0, 1), C(1, 0)) + p_{1,1} = \max(0, 0) + 5 = 5$$

$$C(J2, M1) = C(2, 1) = \max(C(1, 1), C(2, 0)) + p_{2,1} = \max(5, 0) + 2 = 7$$

$$C(J3, M1) = C(3, 1) = \max(C(2, 1), C(3, 0)) + p_{3,1} = \max(7, 0) + 4 = 11$$

Máquina 2 ($j = 2$): cada trabajo espera lo que sea mayor entre ($M2$ libre) y (el trabajo ya salió de $M1$).

$$C(J1, M2) = C(1, 2) = \max(C(0, 2), C(1, 1)) + p_{1,2} = \max(0, 5) + 3 = 8$$

$$C(J2, M2) = C(2, 2) = \max(C(1, 2), C(2, 1)) + p_{2,2} = \max(8, 7) + 6 = 14$$

$$C(J3, M2) = C(3, 2) = \max(C(2, 2), C(3, 1)) + p_{3,2} = \max(14, 11) + 4 = 18$$

Makespan de esta secuencia: $C_{max} = 18$ (el último valor calculado, en la última máquina).

Noten el patrón: para calcular $C(i, j)$ siempre necesitan el valor de la celda de arriba ($C(i-1, j)$) y el de la celda de la izquierda ($C(i, j-1)$) — por eso en código conviene llenar esta tabla como una matriz, recorriendo primero los trabajos de la secuencia y, para cada uno, las máquinas en orden.

Este es exactamente el fitness que va a evaluar tu Algoritmo Genético: dado un cromosoma (una permutación), se recorre esta tabla y el makespan resultante es el valor que hay que **minimizar**.

Al probar con otra secuencia, por ejemplo $\pi = (J2, J1, J3)$, se va a obtener un makespan distinto — ese es justamente el espacio de búsqueda que explora el AG.

4. Segundo método a implementar (comparación)

Además del AG, deben implementar **un Algoritmo Memético**: el mismo AG descrito arriba, pero agregando una etapa de **búsqueda local** después del cruce/mutación (o cada cierto número de generaciones) aplicada al mejor individuo o a toda la población. Una opción simple y efectiva para permutaciones es el operador **insertion local search** o **2-opt adaptado a permutaciones**: probar mover cada trabajo a otras posiciones cercanas y quedarse con la mejora si la hay.

Esto les permitirá comparar **AG puro (exploración)** vs **AG + búsqueda local (exploración + explotación)**.

5. Dataset: Instancias de Taillard

Usen las instancias estándar de la literatura, con óptimos/mejores valores conocidos publicados, disponibles en:

- [Repositorio 1](#)
- [Repositorio 2](#)
- [Mejores Resultados](#)
- [Artículo Estado del Arte](#)

Formato de los archivos

Cada archivo contiene varias instancias. Para cada una encontrarán:

- Una línea con **number of jobs, number of machines, ...**
- Una matriz de tiempos de procesamiento: **filas = máquinas, columnas = trabajos** (¡atención al orden, es fácil confundirlo al leer el archivo!)

Tamaños recomendados para el proyecto

- Empiecen probando con instancias pequeñas (20 trabajos x 5 máquinas) para validar que su código funciona y que el makespan calculado tiene sentido.
- Luego trabajen con al menos 2-3 tamaños distintos (por ejemplo 20x5, 50x10, 100x10) para poder analizar cómo escala cada método.

Cada instancia de Taillard tiene su **mejor valor conocido (Upper Bound)** publicado, lo que les permite calcular el **gap porcentual** de sus soluciones — la métrica de comparación más usada en la literatura:

$$RPD(\%) = \frac{C_{max}^{obtenido} - C_{max}^{óptimo/mejor conocido}}{C_{max}^{óptimo/mejor conocido}} \times 100$$

(RPD = Relative Percentage Deviation)

6. Trabajo de Investigación (Informe Escrito)

Objetivo: elaborar un artículo científico, siguiendo la plantilla IEEE Transactions, que documente todo el desarrollo del proyecto: marco teórico, diseño experimental, resultados y conclusiones de la comparación **AG vs Algoritmo Memético** aplicados al PFSP. Se busca ejercitar la redacción científica y la capacidad de relacionar teoría, implementación y análisis crítico.

Estructura mínima del artículo:

- Título, autor(es) y afiliación institucional.
- **Resumen:** problema, enfoque metodológico, resultados principales y relevancia del estudio.
- **Introducción:** contexto del *scheduling* industrial, definición del PFSP, breve estado del arte, objetivos específicos.
- **Métodos:**
 - Marco teórico: el problema PFSP y las dos metaheurísticas usadas (AG y Memético).
 - Modelamiento y diseño: representación, función de fitness, operadores genéticos elegidos y justificación, operador de búsqueda local del Memético, mecanismo de reemplazo/elitismo y criterio de término.
- **Resultados:** diseño experimental, ajuste y justificación de parámetros, tablas/gráficas comparando AG vs Memético, test estadístico.
- **Conclusiones:** principales aportes, ventajas y limitaciones de cada método, posibles líneas de trabajo futuro.
- **Referencias** en formato IEEE (incluir fuentes relevantes y actuales).

Extensión: 2000-3000 palabras. El trabajo debe ser **100% original** (no copiado de internet ni de otra fuente) — de lo contrario se evalúa con nota NCR.

Evaluación: Informe final (rúbrica del trabajo escrito) 70% — Avance semanal en Overleaf o equivalente 30%.

Entrega: redactar en Overleaf o equivalente y subir el documento final en EVA. Fecha según Syllabus o acuerdo en clases.

7. Trabajo de Programación (Implementación)

Enunciado: desarrollar en Python 3 un programa que resuelva el PFSP implementando **(a) Algoritmo Genético** y **(b) Algoritmo Memético**. La solución se modela como un cromosoma tipo permutación: un vector de n enteros.

Funciones mínimas que debe tener el código:

- Generar un número real aleatorio en $[0, 1]$.
- Generar un número entero aleatorio en un rango dado.
- Leer y parsear una instancia de Taillard (matriz de tiempos $p_{i,j}$).
- Inicializar la población.
- Calcular el fitness de un individuo.
- Seleccionar un individuo.
- Cruzar dos individuos con un operador válido.
- Mutar un individuo.
- Aplicar búsqueda local sobre un individuo (variante Memética).
- Reemplazar/reducir la población, aplicando elitismo (opcional).

Parámetros que debe recibir el programa:

- Semilla aleatoria.
- Instancia a resolver.
- Tamaño de la población.
- Probabilidad de cruce.
- Probabilidad de mutación.
- Número de generaciones/iteraciones.
- (Memético) frecuencia o intensidad de la búsqueda local.

Metodología de desarrollo: programación modular — los operadores genéticos deben implementarse como funciones/métodos genéricos y reutilizables, formando una pequeña "biblioteca" aplicable a futuros proyectos. Se recomienda usar control de versiones (GitHub).

Evaluación:

- Interfaz (presentación por pantalla, manejo de errores, orden): 20%
- Código (originalidad, uso de funciones y estructuras de datos): 30%
- Funcionalidad (nivel de cumplimiento de los requerimientos): 50%

El trabajo debe ser 100% original (no copiado de internet ni de un compañero) — de lo contrario, nota NCR. El programa será testeado en vivo con el profesor para verificar su correcta codificación y ejecución.

Entrega: comprimir la carpeta del proyecto y subirla a EVA. Fecha según Syllabus o acuerdo en clases.

8. Referencias base para investigar el problema

- Taillard, E. (1993). *Benchmarks for basic scheduling problems*. European Journal of Operational Research.
- Reeves, C. R. (1995). *A genetic algorithm for flowshop sequencing*. Computers & Operations Research.